

COAL

Lecture 6

Bit Masking

- Bit masking involves using binary masks to manipulate specific bits within a binary number through bitwise operations.
 - Type of bit manipulation
 - Binary masks are the bitwise operations
 - AND, OR, NOT, XOR
- ```
MOV AL, 10101010b ; value
AND AL, 00001111b ; mask to extract the lower 4 bits
; Result in AL: 00001010b
```



# Bit masking

---

- Selective set -> set a specific bit (1)
- Selective clear -> clear a specific bit (0)
- Selective compliment -> Compliment of a specific bit (1->0, 0 ->1)



# Selective Set

---

- A selective set operation sets specific bits to 1 in a binary value without changing the other bits.
- OR operation is used with **mask** to set bits in original value to 1.
- Rest of bits remain 0.
- Let's set the 2nd and 4th bits of a binary number **1010 0000b**, by using mask **00001010b**
  - $10100000b \mid 00001010b \rightarrow 10101010b$



# Selective Set

---

- Identify mask to set 2<sup>nd</sup> and 6<sup>th</sup> bit in 10010000b.
  - 00100010b
- Identify mask to set 1<sup>st</sup>, 3<sup>rd</sup> and 4<sup>th</sup> bit in 00000000b.
  - 00001101b
- Which bits will be set in the binary number 00111100b, using mask 01001000b?
  - 7<sup>th</sup> bit



# Selective Clear

---

- A selective clear operation clears specific bits (sets them to 0) without changing the other bits.
- AND operation is used with the mask to clear bits to 0.
- Rest of the bits remain 1.
- To clear the 3rd and 6th bits of a binary number 11101111b, use a mask 11011011b
  - 11101111b & 11011011b -> 11001011b



# Selective Clear

---

- Identify mask to clear 3<sup>rd</sup> and 7<sup>th</sup> bit in 11101111b.
  - 10111011b
- Identify mask to clear 1<sup>st</sup>, 6<sup>th</sup> and 8<sup>th</sup> bit in 10101011b.
  - 01011110b
- Which bits will be cleared in the binary number 10101100b, using mask 01111011b?
  - 8<sup>th</sup> bit and 3<sup>rd</sup> bit



# Selective Compliment

---

- Flipping the bits( 0 to 1 and 1 to 0) at specific positions in a binary number, based on a mask.
- Selectively inverts bits according to where the mask has 1.
- XOR operation (^) is used with value and mask.
- Flip 1<sup>st</sup>, 5<sup>th</sup>, 7<sup>th</sup> and 8<sup>th</sup> bit in **10101100b**, using mask 11010001b
  - **01111101b**                      1->1 = 0; 0->0 = 0      ;    1->0=1, 0->1=1



# Selective Compliment

---

- Identify mask to flip 3<sup>rd</sup>, 5<sup>th</sup> and 7<sup>th</sup> bit in 11101111b.
  - 01010100b
- Identify mask to flip 1<sup>st</sup>, 6<sup>th</sup> and 8<sup>th</sup> bit in 10101011b.
  - 10100001b
- Which bits will be flipped in the binary number 10101100b, using mask 01111011b?
  - 1, 2, 4, 5, 6, 7



# Stack

---

- Stack is a data structure used for managing function calls, local variables, and control flow.
- Last in First Out (LIFO)
  - Last item pushed onto the stack is the first one to be popped off.
- **.stack** directive is used to reserve a block of memory for stack. (.stack 100h)
- SS (Stack Segment) Register: Holds stack Segment Address
- The stack pointer (SP) register is used to keep track of the top of the stack.



# Operations on Stack

---

- Push: Adds an item to the top of the stack.
- Pop: Removes the top item from the stack.
- Peek: Views the top item without removing it.



# Push and PushF

---

- Push: Used to add a new source (16-bit register or memory word) on stack

- Syntax:

*PUSH source*

- Execution of PUSH causes:
  - SP is decreased by 2 (stack grows downward in memory)
  - A copy of source contents is moved to the address specified by SS:SP.
  - Source remains unchanged
- PUSHF has no operand and it pushes the contents of FLAG register onto the stack. <sup>12</sup>



# PUSH - Example

| Offset | Stack |
|--------|-------|
| 00F8   |       |
| 00FA   |       |
| 00FC   |       |
| 00FE   |       |
| 0100   | ← SP  |

|    |      |
|----|------|
| SP | 0100 |
| AX | 1234 |
| BX | 5678 |

***INITIAL STACK***

| Offset | Stack     |
|--------|-----------|
| 00F8   |           |
| 00FA   |           |
| 00FC   |           |
| 00FE   | 1234 ← SP |
| 0100   |           |

|    |      |
|----|------|
| SP | 00FE |
| AX | 1234 |
| BX | 5678 |

***After PUSH AX***

| Offset | Stack     |
|--------|-----------|
| 00F8   |           |
| 00FA   |           |
| 00FC   | 5678 ← SP |
| 00FE   | 1234      |
| 0100   |           |

|    |      |
|----|------|
| SP | 00FC |
| AX | 1234 |
| BX | 5678 |

***After PUSH BX***



# POP And POPF

---

- POP: Used to remove top item from stack to destination (i.e. a 16-bit register or memory word).
- Syntax:  
*POP destination*
- Execution of POP causes:
  - The contents of SS:SP (top of the stack) is moved to the destination.
  - SP is increased by 2
- POPF has no operand and pops the top of the stack into FLAG register.



# POP - Example

| Offset | Stack     |
|--------|-----------|
| 00F8   |           |
| 00FA   |           |
| 00FC   | 5678 ← SP |
| 00FE   | 1234      |
| 0100   |           |

|    |      |
|----|------|
| SP | 00FC |
| CX | FFFF |
| DX | 0001 |

***Before POP***

| Offset | Stack     |
|--------|-----------|
| 00F8   |           |
| 00FA   |           |
| 00FC   |           |
| 00FE   | 1234 ← SP |
| 0100   |           |

|    |      |
|----|------|
| SP | 00FE |
| CX | 5678 |
| DX | 0001 |

***After POP CX***

| Offset | Stack |
|--------|-------|
| 00F8   |       |
| 00FA   |       |
| 00FC   |       |
| 00FE   |       |
| 0100   |       |

|    |      |
|----|------|
| SP | 0100 |
| CX | 5678 |
| DX | 1234 |

***After POP DX  
(Stack Empty)***



# Uses of Stack

---

- Function Calls: When a function is called, the return address is pushed onto the stack, and local variables are stored there.
- Interrupts: The state of the CPU (registers) is saved onto the stack when an interrupt occurs, allowing the program to resume afterward.
- Context Switching: In multitasking systems, the stack saves the state of a process before switching to another process.
- To reverse an input



# Example

---

```
PUSH AX ; Save the value in AX register to the stack
POP BX ; Pop the top value from the stack into BX register
```

**PUSH:** Decreases the stack pointer and stores data.

**POP:** Increases the stack pointer and retrieves data.



# Saving and Restoring Variable in Stack

---

- Technique used to:
  - Manage function calls
  - Local variables
  - Preserve register values
- Allows the program to save the current state before making changes and restore it afterward, ensuring that the program can continue execution as if nothing was altered.



# How it works?

---

- Saving a Variable:
  - Before modifying a register or variable, you push its value onto the stack. This ensures you can restore it later.
- Restoring a Variable:
  - After you're done using the register or modifying the variable, you pop the value back from the stack, restoring the original state.



# Example

---

; Saving a Variable (AX)

PUSH AX           ; Save the value in AX register onto the stack

MOV AX, 10       ; Change AX to a new value (for example, 10)

; Do some operations here using the modified AX value

; Restoring the Variable (AX)

POP AX           ; Restore the original value of AX from the stack

; Now, AX holds the original value again (before it was modified)

# Example

---

```
; Example: Function call with saved registers
PUSH AX ; Save AX register before calling the function
PUSH BX ; Save BX register before calling the function

CALL MyFunction ; Call the function

POP BX ; Restore BX after returning from the function
POP AX ; Restore AX after returning from the function

; Continue execution with original AX and BX values restored
```



# Accessing Stack with BP

---

- Base Pointer Register – points towards the stack base
- No pop instruction is required
- Contents of the SP register are moved to the BP register to gain access to the top of the stack.
- Then, using register indirect mode, the value anywhere in the stack can be read or written.



# Example

---

**Example:**

Accessing top of the stack

```
Mov BP, SP
MOV AX, [BP]
```

**Example:**

Accessing the second value from the top of the stack

```
Mov BP, SP
MOV AX, [BP+2]
```

**Example:**

Accessing the third value from the top of the stack

```
Mov BP, SP
MOV AX, [BP+4]
```